

Otimização

3.2.4 Método de Box — versão simplificada de EVOP (*evolutionary operation*)

José Firmino Aguilar Madeira

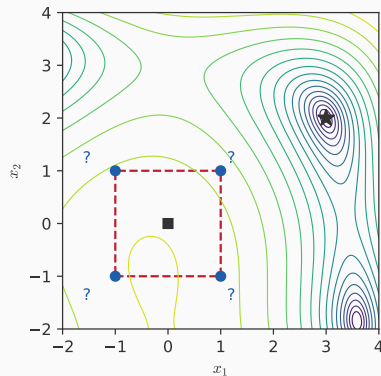
Do simplex que muda de forma para uma caixa que não muda

O Nelder–Mead **adapta a forma** do simplex usando os valores de f . E se mantivermos sempre o **mesmo padrão geométrico**?

Em 2D, à volta do ponto atual x , os quatro vértices de uma caixa:

$$x + \frac{1}{2}(\pm\Delta_1, \pm\Delta_2).$$

Aqui $x = (0, 0)$, $\Delta = (2, 2)$, no Himmelblau.



Do simplex que muda de forma para uma caixa que não muda

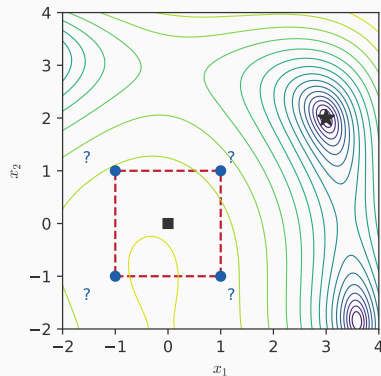
O Nelder–Mead **adapta a forma** do simplex usando os valores de f . E se mantivermos sempre o **mesmo padrão geométrico**?

Em 2D, à volta do ponto atual x , os quatro vértices de uma caixa:

$$x + \frac{1}{2}(\pm\Delta_1, \pm\Delta_2).$$

Aqui $x = (0, 0)$, $\Delta = (2, 2)$, no Himmelblau.

Qual deles escolheríamos?



Do simplex que muda de forma para uma caixa que não muda

O Nelder–Mead **adapta a forma** do simplex usando os valores de f . E se mantivermos sempre o **mesmo padrão geométrico**?

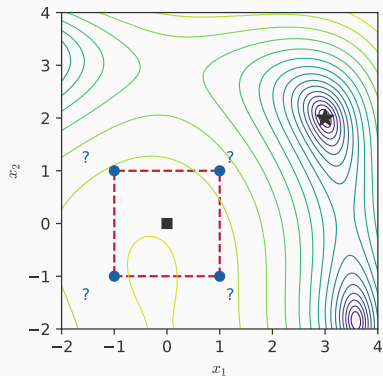
Em 2D, à volta do ponto atual x , os quatro vértices de uma caixa:

$$x + \frac{1}{2}(\pm\Delta_1, \pm\Delta_2).$$

Aqui $x = (0, 0)$, $\Delta = (2, 2)$, no Himmelblau.

Qual deles escolheríamos?

→ o de **menor f** — se melhorar o centro.



Do simplex que muda de forma para uma caixa que não muda

O Nelder–Mead **adapta a forma** do simplex usando os valores de f . E se mantivermos sempre o **mesmo padrão geométrico**?

Em 2D, à volta do ponto atual x , os quatro vértices de uma caixa:

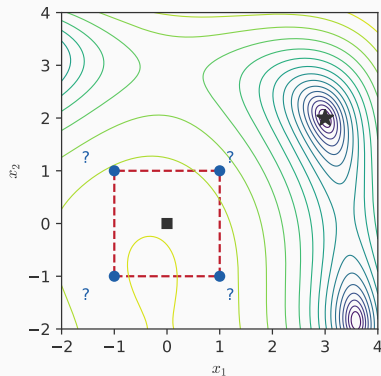
$$x + \frac{1}{2}(\pm\Delta_1, \pm\Delta_2).$$

Aqui $x = (0, 0)$, $\Delta = (2, 2)$, no Himmelblau.

Qual deles escolheríamos?

→ o de **menor f** — se melhorar o centro.

pesquisa aleatória (3.2.1)	pontos sem estrutura
grelha (3.2.2)	estrutura fixa, no domínio todo
Nelder–Mead (3.2.3)	geometria adaptativa , que se move
Box (3.2.4)	padrão fixo , que se move

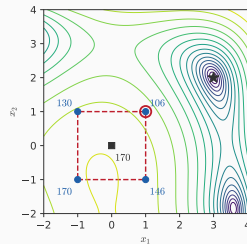


Uma iteração à mão

$$f(x) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2, \quad x = (0, 0),$$

$$f(x) = 170, \quad \Delta = (2, 2).$$

vértice	f
$(-1, -1)$	170
$(1, -1)$	146
$(-1, 1)$	130
$(1, 1)$	106



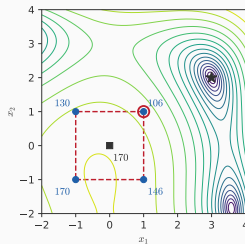
Uma iteração à mão

$$f(x) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2, \quad x = (0, 0),$$

$$f(x) = 170, \quad \Delta = (2, 2).$$

vértice	f
$(-1, -1)$	170
$(1, -1)$	146
$(-1, 1)$	130
$(1, 1)$	106

O melhor, $f(1, 1) = 106 < 170$: $x \leftarrow (1, 1)$, e Δ fica igual.



Uma iteração à mão

$$f(x) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2, \quad x = (0, 0),$$

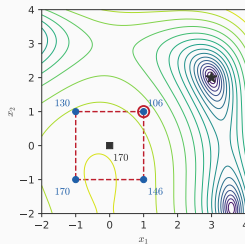
$$f(x) = 170, \quad \Delta = (2, 2).$$

vértice	f
$(-1, -1)$	170
$(1, -1)$	146
$(-1, 1)$	130
$(1, 1)$	106

O melhor, $f(1, 1) = 106 < 170$: $x \leftarrow (1, 1)$, e Δ fica igual.

E se nenhum dos quatro fosse melhor?

$$\Delta \leftarrow \Delta/2$$



Uma iteração à mão

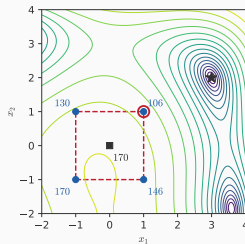
$$f(x) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2, \quad x = (0, 0),$$
$$f(x) = 170, \quad \Delta = (2, 2).$$

vértice	f
$(-1, -1)$	170
$(1, -1)$	146
$(-1, 1)$	130
$(1, 1)$	106

O melhor, $f(1, 1) = 106 < 170$: $x \leftarrow (1, 1)$, e Δ fica igual.

E se nenhum dos quatro fosse melhor? $\Delta \leftarrow \Delta/2$

Acabámos de construir o algoritmo inteiro.



Origem

G. E. P. Box (1957), *Evolutionary Operation*: afinar processos industriais *em produção* (Box trabalhava na ICI), com pequenas variações em torno das condições atuais; cada avaliação era um lote. O EVOP original é experimental e estatístico; aqui usa-se uma **versão determinística simplificada** (não confundir com o método *Complex* de M. J. Box, 1965 — outro autor, outro algoritmo).

O algoritmo, e o seu custo

Box: ponto inicial x_0 , lados Δ_i ,
tolerância ε_x

```
1:  $x \leftarrow x_0$ ;  $f_x \leftarrow f(x)$ 
2: while  $\|\Delta\| \geq \varepsilon_x$  do
3:   gerar os  $2^n$  vértices  $x + \frac{1}{2}(\pm\Delta_1, \dots, \pm\Delta_n)$ 
4:   avaliar  $f$  em todos;  $x' \leftarrow$  o melhor
5:   if  $f(x') < f_x$  then  $x \leftarrow x'$ ;  $f_x \leftarrow f(x')$   $\triangleright$  mover
6:   else  $\Delta \leftarrow \Delta/2$   $\triangleright$  encolher
7:   end if
8: end while
9: return  $x$ 
```

- Os Δ_i podem ser diferentes por coordenada; paragem quando a caixa fica menor do que ε_x .
- Uma implementação com *cache* pode reaproveitar pontos já avaliados; nas contagens deste deck avaliam-se sempre os 2^n vértices.

$$n_f = 2^n \text{ por iteração}$$

n	2	3	10	20
vértices	4	8	1 024	$\approx 10^6$

O algoritmo, e o seu custo

Box: ponto inicial x_0 , lados Δ_i ,
tolerância ε_x

```
1:  $x \leftarrow x_0$ ;  $f_x \leftarrow f(x)$ 
2: while  $\|\Delta\| \geq \varepsilon_x$  do
3:   gerar os  $2^n$  vértices  $x + \frac{1}{2}(\pm\Delta_1, \dots, \pm\Delta_n)$ 
4:   avaliar  $f$  em todos;  $x' \leftarrow$  o melhor
5:   if  $f(x') < f_x$  then  $x \leftarrow x'$ ;  $f_x \leftarrow f(x')$   $\triangleright$  mover
6:   else  $\Delta \leftarrow \Delta/2$   $\triangleright$  encolher
7:   end if
8: end while
9: return  $x$ 
```

- Os Δ_i podem ser diferentes por coordenada; paragem quando a caixa fica menor do que ε_x .
- Uma implementação com *cache* pode reaproveitar pontos já avaliados; nas contagens deste deck avaliam-se sempre os 2^n vértices.

$$n_f = 2^n \text{ por iteração}$$

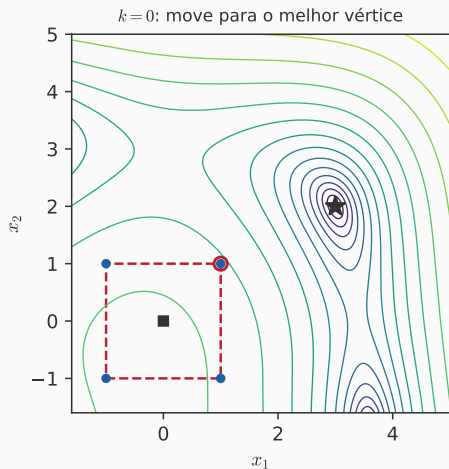
n	2	3	10	20
vértices	4	8	1 024	$\approx 10^6$

A maldição da dimensão outra vez — por uma razão diferente:

- pesquisa aleatória e grelha (3.2.1–3.2.2): o **volume** a cobrir;
- Box: o **número de vértices** a testar.

Mesma consequência: a dimensão custa muito.

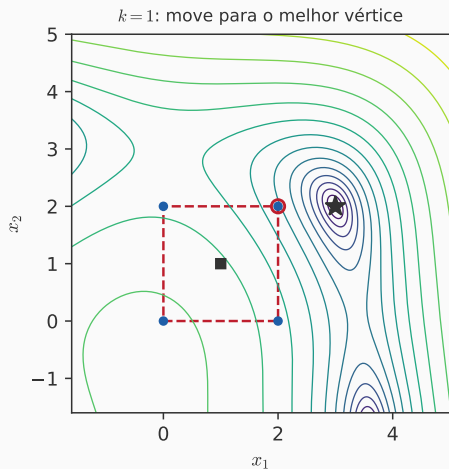
Himmelblau a partir de $(0,0)$: mover ou encolher



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f_{novo}
0	(0,000; 0,000)	2,000	170,000	move	106,000

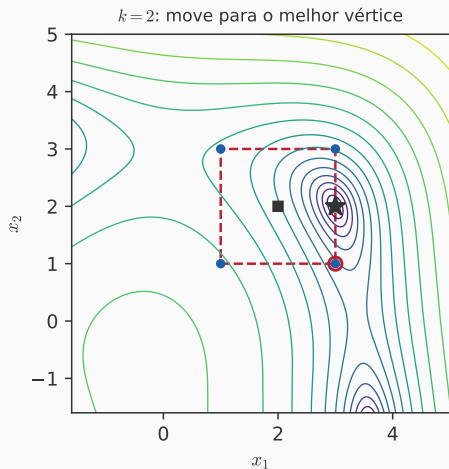
Himmelblau a partir de $(0,0)$: mover ou encolher



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f novo
0	(0,000; 0,000)	2,000	170,000	move	106,000
1	(1,000; 1,000)	2,000	106,000	move	26,000

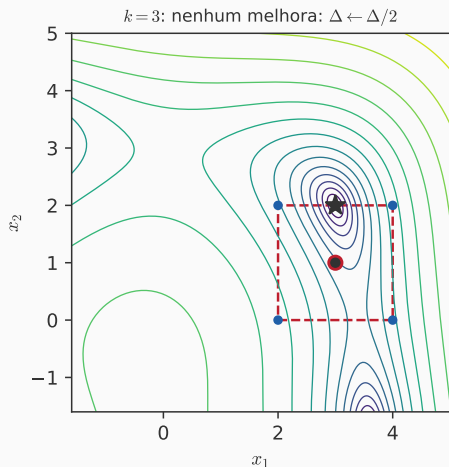
Himmelblau a partir de (0,0): mover ou encolher



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f novo
0	(0,000; 0,000)	2,000	170,000	move	106,000
1	(1,000; 1,000)	2,000	106,000	move	26,000
2	(2,000; 2,000)	2,000	26,000	move	10,000

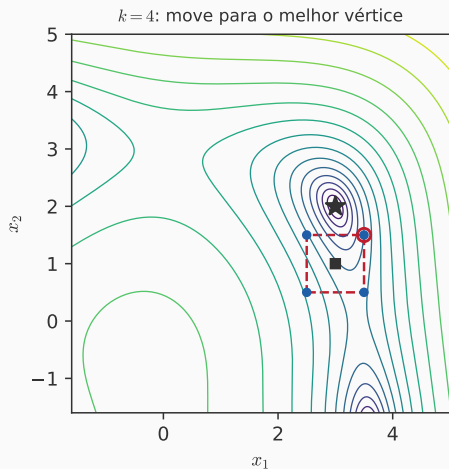
Himmelblau a partir de $(0,0)$: mover ou encolher



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f novo
0	(0,000; 0,000)	2,000	170,000	move	106,000
1	(1,000; 1,000)	2,000	106,000	move	26,000
2	(2,000; 2,000)	2,000	26,000	move	10,000
3	(3,000; 1,000)	2,000	10,000	$\Delta \leftarrow \Delta/2$	10,000

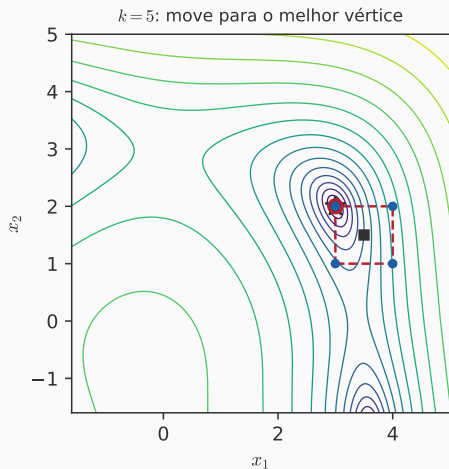
Himmelblau a partir de (0,0): mover ou encolher



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f novo
0	(0,000; 0,000)	2,000	170,000	move	106,000
1	(1,000; 1,000)	2,000	106,000	move	26,000
2	(2,000; 2,000)	2,000	26,000	move	10,000
3	(3,000; 1,000)	2,000	10,000	$\Delta \leftarrow \Delta/2$	10,000
4	(3,000; 1,000)	1,000	10,000	move	9,125

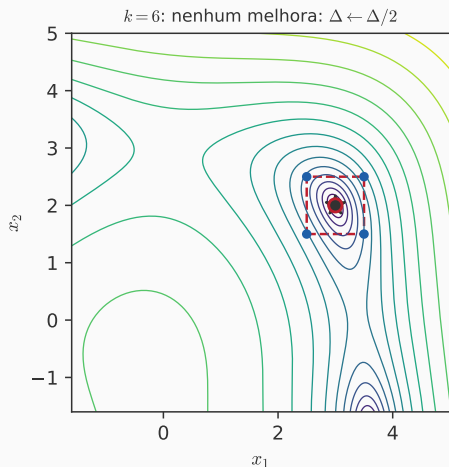
Himmelblau a partir de (0,0): mover ou encolher



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f novo
0	(0,000; 0,000)	2,000	170,000	move	106,000
1	(1,000; 1,000)	2,000	106,000	move	26,000
2	(2,000; 2,000)	2,000	26,000	move	10,000
3	(3,000; 1,000)	2,000	10,000	$\Delta \leftarrow \Delta/2$	10,000
4	(3,000; 1,000)	1,000	10,000	move	9,125
5	(3,500; 1,500)	1,000	9,125	move	0,000

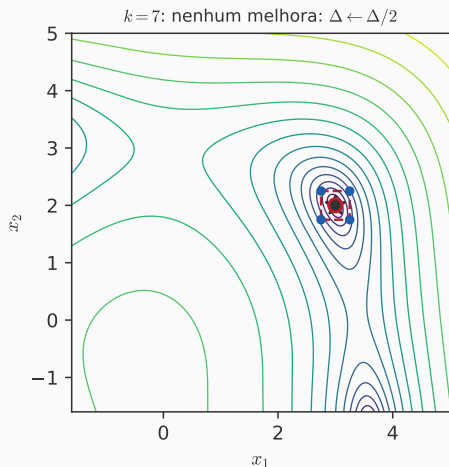
Himmelblau a partir de (0,0): mover ou encolher



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f_{novo}
0	(0,000; 0,000)	2,000	170,000	move	106,000
1	(1,000; 1,000)	2,000	106,000	move	26,000
2	(2,000; 2,000)	2,000	26,000	move	10,000
3	(3,000; 1,000)	2,000	10,000	$\Delta \leftarrow \Delta/2$	10,000
4	(3,000; 1,000)	1,000	10,000	move	9,125
5	(3,500; 1,500)	1,000	9,125	move	0,000
6	(3,000; 2,000)	1,000	0,000	$\Delta \leftarrow \Delta/2$	0,000

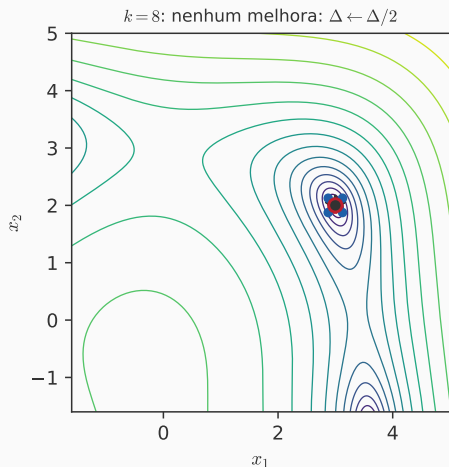
Himmelblau a partir de (0,0): mover ou encolher



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f novo
0	(0,000; 0,000)	2,000	170,000	move	106,000
1	(1,000; 1,000)	2,000	106,000	move	26,000
2	(2,000; 2,000)	2,000	26,000	move	10,000
3	(3,000; 1,000)	2,000	10,000	$\Delta \leftarrow \Delta/2$	10,000
4	(3,000; 1,000)	1,000	10,000	move	9,125
5	(3,500; 1,500)	1,000	9,125	move	0,000
6	(3,000; 2,000)	1,000	0,000	$\Delta \leftarrow \Delta/2$	0,000
7	(3,000; 2,000)	0,500	0,000	$\Delta \leftarrow \Delta/2$	0,000

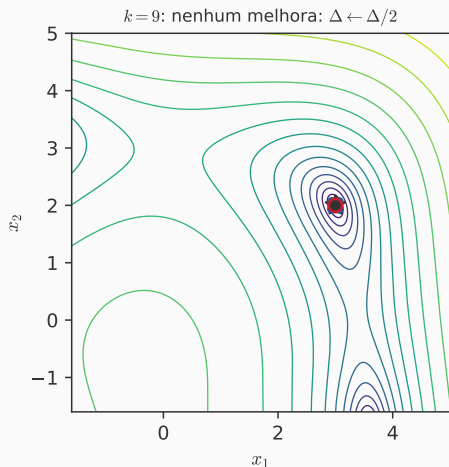
Himmelblau a partir de (0,0): mover ou encolher



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f novo
0	(0,000; 0,000)	2,000	170,000	move	106,000
1	(1,000; 1,000)	2,000	106,000	move	26,000
2	(2,000; 2,000)	2,000	26,000	move	10,000
3	(3,000; 1,000)	2,000	10,000	$\Delta \leftarrow \Delta/2$	10,000
4	(3,000; 1,000)	1,000	10,000	move	9,125
5	(3,500; 1,500)	1,000	9,125	move	0,000
6	(3,000; 2,000)	1,000	0,000	$\Delta \leftarrow \Delta/2$	0,000
7	(3,000; 2,000)	0,500	0,000	$\Delta \leftarrow \Delta/2$	0,000
8	(3,000; 2,000)	0,250	0,000	$\Delta \leftarrow \Delta/2$	0,000

Himmelblau a partir de (0,0): mover ou encolher

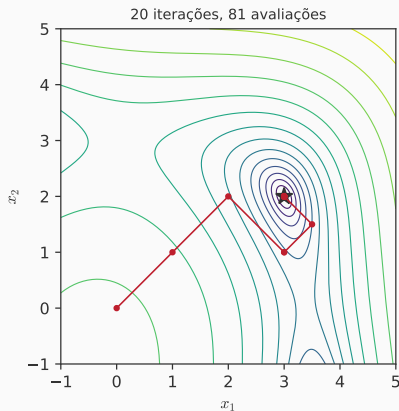


$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. Em cada linha, o centro x_k e a decisão tomada a partir dele.

k	x_k	Δ	$f(x_k)$	decisão	f novo
0	(0,000; 0,000)	2,000	170,000	move	106,000
1	(1,000; 1,000)	2,000	106,000	move	26,000
2	(2,000; 2,000)	2,000	26,000	move	10,000
3	(3,000; 1,000)	2,000	10,000	$\Delta \leftarrow \Delta/2$	10,000
4	(3,000; 1,000)	1,000	10,000	move	9,125
5	(3,500; 1,500)	1,000	9,125	move	0,000
6	(3,000; 2,000)	1,000	0,000	$\Delta \leftarrow \Delta/2$	0,000
7	(3,000; 2,000)	0,500	0,000	$\Delta \leftarrow \Delta/2$	0,000
8	(3,000; 2,000)	0,250	0,000	$\Delta \leftarrow \Delta/2$	0,000
9	(3,000; 2,000)	0,125	0,000	$\Delta \leftarrow \Delta/2$	0,000

A caixa **desloca-se** com tamanho fixo,
 $(0, 0) \rightarrow (1, 1) \rightarrow (2, 2) \rightarrow (3, 1)$; em $(3, 1)$ nenhum
vértice melhora e **encolhe** ($\Delta: 2 \rightarrow 1$); depois
 $(3, 1) \rightarrow (3,5; 1,5) \rightarrow (3, 2)$ e só encolhe.

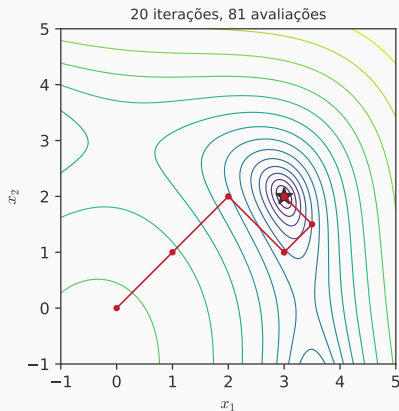
O percurso completo: o que a caixa está a fazer



20 iterações, $n_f = 4 \times 20 + 1 = 81$ até
 $\|\Delta\| < 10^{-4}$.

- Enquanto há melhoria, o centro salta de vértice em vértice: todas as coordenadas mudam $\pm\Delta_i/2$, logo o passo tem comprimento fixo $\frac{1}{2} \|\Delta\|$. Só quando estagna é que diminui.
- Chega a (3,2) *exatamente* porque o mínimo está sobre a grelha de passos $\Delta/2$.

O percurso completo: o que a caixa está a fazer

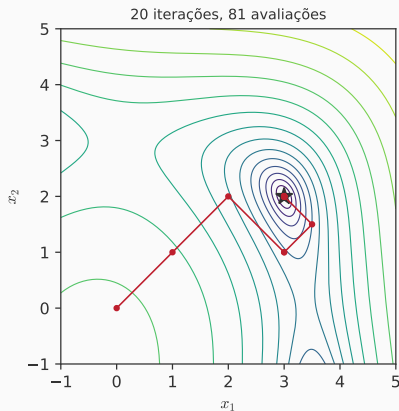


20 iterações, $n_f = 4 \times 20 + 1 = 81$ até
 $\|\Delta\| < 10^{-4}$.

- Enquanto há melhoria, o centro salta de vértice em vértice: todas as coordenadas mudam $\pm\Delta_i/2$, logo o passo tem comprimento fixo $\frac{1}{2} \|\Delta\|$. Só quando estagna é que diminui.
- Chega a $(3, 2)$ *exatamente* porque o mínimo está sobre a grelha de passos $\Delta/2$.

Se Δ é muito pequeno, sabemos que estamos perto de x^* ?

O percurso completo: o que a caixa está a fazer



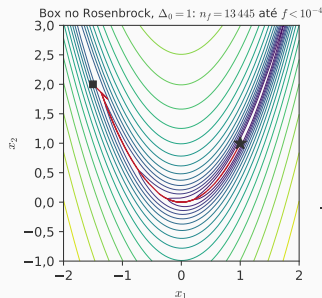
20 iterações, $n_f = 4 \times 20 + 1 = 81$ até
 $\|\Delta\| < 10^{-4}$.

- Enquanto há melhoria, o centro salta de vértice em vértice: todas as coordenadas mudam $\pm\Delta_i/2$, logo o passo tem comprimento fixo $\frac{1}{2} \|\Delta\|$. Só quando estagna é que diminui.
- Chega a $(3, 2)$ *exatamente* porque o mínimo está sobre a grelha de passos $\Delta/2$.

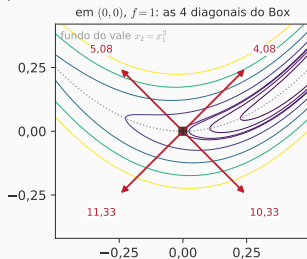
Se Δ é muito pequeno, sabemos que estamos perto de x^* ?

Não. Sabemos apenas que, naquele padrão e àquela escala, nenhum vértice testado melhorou. Uma caixa pequena não certifica estacionariedade a Δ finito, nem mínimo local, nem $\|x - x^*\| = O(\varepsilon_x)$.

Porque falha em vales curvos?

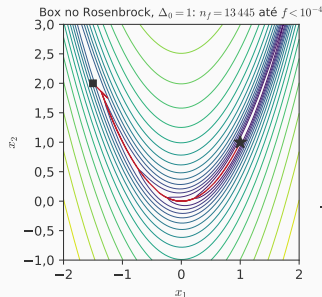


Qual é o problema de mudar todas as coordenadas ao mesmo tempo? No Rosenbrock, a partir de $(-1,5; 2)$ com $\Delta_0 = 1$, a caixa tem de encolher de 1 até $\approx 6 \cdot 10^{-5}$ para conseguir progredir no vale, e depois avança em passos minúsculos: $n_f = 13\,445$ até $f < 10^{-4}$ (com $\varepsilon_x = 10^{-6}$).

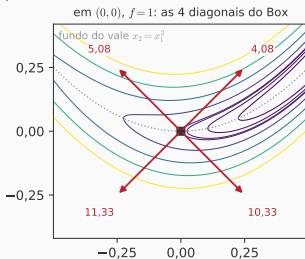


Em $(0,0)$, com passo 0,25, nenhuma das 4 diagonais $(\pm 1, \pm 1)$ melhora $f = 1$ (4,08 a 11,33).

Porque falha em vales curvos?



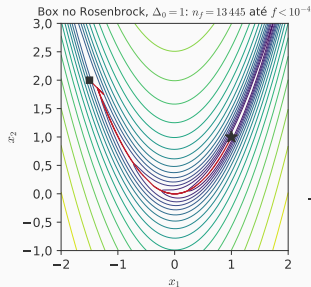
Qual é o problema de mudar todas as coordenadas ao mesmo tempo? No Rosenbrock, a partir de $(-1,5; 2)$ com $\Delta_0 = 1$, a caixa tem de encolher de 1 até $\approx 6 \cdot 10^{-5}$ para conseguir progredir no vale, e depois avança em passos minúsculos: $n_f = 13\,445$ até $f < 10^{-4}$ (com $\varepsilon_x = 10^{-6}$).



Em $(0,0)$, com passo 0,25, nenhuma das 4 diagonais $(\pm 1, \pm 1)$ melhora $f = 1$ (4,08 a 11,33).

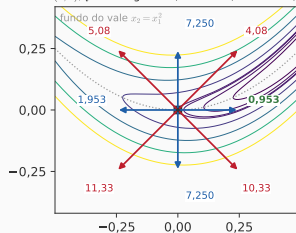
O Box concluiu que não há melhoria. Será que realmente não há uma direção de descida?

Porque falha em vales curvos?



Qual é o problema de mudar todas as coordenadas ao mesmo tempo? No Rosenbrock, a partir de $(-1,5; 2)$ com $\Delta_0 = 1$, a caixa tem de encolher de 1 até $\approx 6 \cdot 10^{-5}$ para conseguir progredir no vale, e depois avança em passos minúsculos: $n_f = 13445$ até $f < 10^{-4}$ (com $\varepsilon_x = 10^{-6}$).

em $(0, 0)$, $f = 1$: diagonais (vermelho) vs. eixos (azul)

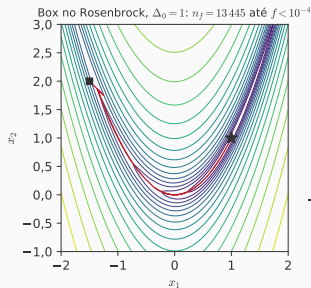


Em $(0, 0)$, com passo 0,25, nenhuma das 4 diagonais $(\pm 1, \pm 1)$ melhora $f = 1$ (4,08 a 11,33).

O Box concluiu que não há melhoria. Será que realmente não há uma direção de descida?

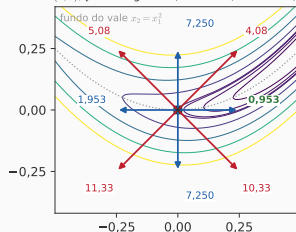
Há: o eixo $+e_1$ melhora, $f(0,25; 0) = 0,953$. O vale é horizontal aí; as diagonais saem dele.

Porque falha em vales curvos?



Qual é o problema de mudar todas as coordenadas ao mesmo tempo? No Rosenbrock, a partir de $(-1,5; 2)$ com $\Delta_0 = 1$, a caixa tem de encolher de 1 até $\approx 6 \cdot 10^{-5}$ para conseguir progredir no vale, e depois avança em passos minúsculos: $n_f = 13445$ até $f < 10^{-4}$ (com $\varepsilon_x = 10^{-6}$).

em $(0, 0)$, $f = 1$: diagonais (vermelho) vs. eixos (azul)



Em $(0, 0)$, com passo 0,25, nenhuma das 4 diagonais $(\pm 1, \pm 1)$ melhora $f = 1$ (4,08 a 11,33).

O Box concluiu que não há melhoria. Será que realmente não há uma direção de descida?

Há: o eixo $+e_1$ melhora, $f(0,25; 0) = 0,953$. O vale é horizontal aí; as diagonais saem dele.

diagonais $(\pm 1, \dots, \pm 1)$ vs. eixos $\pm e_i$

O preço do Box; no computador

Vantagens

ordem zero; determinístico; muito simples; escala Δ ; controlável por variável; as avaliações dos 2^n vértices podem ser feitas em paralelo.

Limitações

- 2^n avaliações por iteração;
- só as 2^n direções diagonais $D = \{(\pm 1, \dots, \pm 1)\}$: depende da orientação das coordenadas;
- ineficiente em vales estreitos e curvos; local;
- caixa pequena $\nrightarrow$ estacionariedade (a Δ finito);
- o ruído pode alterar a escolha do melhor vértice.

No Rosenbrock a partir de $(-1, 5; 2)$, até $f < 10^{-4}$: Box ($\Delta_0 = 1$)
 $n_f = 13\,445$; Nelder–Mead $n_f = 166$ (mesmo problema, mesmo ponto inicial, mesmo critério; com estes parâmetros).

O núcleo, em MATLAB:

```
S = dec2bin(0:2^n-1) - '0';  
S = 2*S - 1; % linhas de +-1  
V = x + 0.5*S.*Delta; % vertices  
Fv = arrayfun(@f(V(i,:)), 1:2^n);  
[fmin, i] = min(Fv);  
if fmin < fx  
    x = V(i,:); fx = fmin; % mover  
else  
    Delta = Delta/2; % encolher  
end
```

Em 2D, S tem as 4 linhas $(-1, -1)$, $(-1, 1)$, $(1, -1)$, $(1, 1)$.

A função completa (BoxEvo, com info.nfev) está no código complementar da UC.

O QUE DEVE FICAR — 3.2.4

1. O Box usa uma **caixa centrada** no ponto atual e testa todas as 2^n combinações diagonais.
2. Melhoria $\Rightarrow$ **mover** para o melhor vértice; insucesso $\Rightarrow \Delta \leftarrow \Delta/2$.
3. Só usa valores de f , mas o custo, **2^n por iteração**, cresce exponencialmente com a dimensão: testar todas as diagonais é **caro e rígido**.
4. Uma caixa pequena não certifica estacionariedade a Δ finito. Para esta versão simplificada do Box não apresentamos aqui uma garantia geral de estacionariedade.
5. O NM não tem garantia geral; o HJ está na origem da pesquisa em padrão, cujas versões formalizadas (GPS) têm estacionariedade no limite sob hipóteses adequadas (3.2.5).

A seguir (3.2.5): precisamos mesmo das 2^n diagonais? E se explorarmos **uma coordenada de cada vez**, com apenas **$2n$ direções**, e depois aproveitarmos a direção do progresso? — o método de Hooke–Jeeves.

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Box, G. E. P., *Evolutionary operation: a method for increasing industrial productivity*, Journal of the Royal Statistical Society, Series C (Applied Statistics) 6(2):81–101, 1957.
- Deb, K., *Optimization for Engineering Design: Algorithms and Examples*, 2.^a ed., PHI Learning, 2012 — cap. 3.

Recursos

- Código complementar: `BoxEvo` (`info.nfev`).